

GAINE - Tema 3 - Bases de datos documental (MongoDB)

- Modelo de datos
- Operaciones básicas
- Operaciones CRUD en MongoDB
- Agregación
- Índices
- Arquitectura de clustering
 - Sharding
 - Réplicas
- Otras características

Modelo de datos

Formato de documentos de MongoDB)

EXAM=(2025S0.13)

El formato principal de documento de base de datos MongoDB es BSON.

BSON es una representación binaria de JSON con algunas particularidades.

Tipos de datos)

- Numéricos
- Texto
- Fecha
- Array
- Identificador de objeto
- Código JavaScript
- Nulo
- Booleano

Conceptos básicos)

El ID (campo `_id`) se muestra siempre por defecto, por lo que es necesario indicar explícitamente que no debe aparecer.

Factores importantes)

Tipos de relaciones)

EXAM=(2023J2.12)

Tipos de relaciones:

- One-to-one
- One-to-many

- Incrustado
- Con referencias
- Many-to-many

Operaciones básicas

La sintaxis de consultas en MongoDB sigue toma como modelo la de JavaScript.

Operaciones CRUD en MongoDB

Creación en MongoDB

EXAM=(2025J2.13)

La función `insertOne()` inserta un único documento en una colección.

```
db.usuarios.insertOne({ nombre: "Carlos", edad: 25 })
```

La función `insertMany()`...

Lectura en MongoDB

La **función find()** sirve para buscar uno o varios registros en base a unas restricciones. Se suele comparar con una sentencia `SELECT ... WHERE` de SQL.

Su forma general es:

```
db.coleccion.find(condicion, proyeccion)
```

Ejemplo:

```
db.socios.find(
  { edad: { $gt: 18 } },
  { nombre: 1, email: 1, _id: 0 }
)
```

EXAM=(2022J2.8, 2022S0.7)

Cuando se usa la función `find`, si se quiere mostrar solamente un campo se debe indicar explícitamente que el identificador en el campo `_id` también se quiere ocultar mediante `_id: 0`.

EXAM=(2025J2.16)

El operador de expresión `$all` aplica el operador AND.

```
db.clientes.find({suscripciones:{$all:["Netflix","Prime Video"]}})
```

El operador de expresión `$in` aplicaría el operador OR.

```
db.clientes.find({suscripciones:{$in:["Netflix","Prime Video"]}})
```

Actualización en MongoDB

La función `updateOne()`...

La función `updateMany()` modifica todos los documentos que cumplan el filtro a la vez.

```
db.usuarios.updateMany({ activo: false }, { $set: { borrar: true } })
```

Borrado en MongoDB

La función `deleteOne()` borra un único documento en una colección.

```
db.usuarios.deleteOne({ _id: 1 })
```

Agregación

Operaciones de agregación:

- Count
- Distinct
- Aggregate
- MapReduce

Count

EXAM=(2023J2.14)

`count()` cuenta el número de documentos de una colección en los que aparece un valor concreto. Cuando se usa directamente contra el motor de base de datos puede provocar problemas de rendimiento y lecturas sucias. Está obsoleto en favor de `estimatedDocumentCount()` y `countDocument()`.

`estimatedDocumentCount()` es una función que calcula de forma aproximada el número de documentos sin ralentizar el servidor. Supone un sustituto más rápido y seguro de `count()` usado directamente contra el motor de base de datos.

`countDocuments()` (escrita con *camel case*) es una función que cuenta el número de documentos de una colección en los que aparece un valor concreto. No aparece en la presentación general, pero se preguntó en un examen. Es la versión moderna y recomendada ante el tradicional y obsoleto `find().count()`.

No puede quedar sin argumentos, y requiere un filtro explícito, aunque sea vacío:

```
db.webs.countDocuments({})
```

Distinct

Aggregate

EXAM=(2022J2.7,2022J2.10,GAINEX.2023J2.15,2025S0.12,2025S0.15,2025S0.16)

`aggregate` permite realizar consultas complejas.

Con `aggregate` se define una tubería que contiene una o varias etapas de agregación. Estas, a su vez, pueden contener operadores de expresión.

Etapas de agregación con `aggregate`:

- `$project`
- `$match`
- `$limit`
- `$lookup`
- `$skip`
- `$unwind`
- `$group`
- `$sort`
- `$geoNear`
- `$lookup`

La **etapa de agregación `$project`** remodela un documento, permitiendo incluir, excluir, renombrar o transformar campos específicos.

Equivale a `SELECT` de SQL.

EXAM=(2023J2.15,2024S0.13)

La **etapa de agregación `$match`** filtra documentos en la entrada indicada.

Equivale a `WHERE` de SQL.

La etapa de agregación `$limit` reduce el número de documentos de salida.

EXAM=(2025S0.17)

La **etapa de agregación `$lookup`** sirve para enlazar documentos en base a una clave común.

```
db.pedidos.aggregate([
  {
    $lookup: {
      from: "usuarios",           // La colección con la que te quieres unir
      localField: "usuario_id",  // El campo en la colección 'pedidos'
      foreignField: "_id",       // El campo en la colección 'usuarios'
      as: "datos_usuario"       // El nombre del nuevo array que tendrá el resultado
    }
  }
])
```

La **etapa de agregación \$skip** salta el número indicado de registros, devolviendo los siguientes.

```
db.articulos.aggregate([
  { $sort: { fecha: -1 } }, // Primero ordenamos (buena práctica antes de un skip)
  { $skip: 5 }              // Nos saltamos los primeros 5 documentos
])
```

EXAM=(2022J2.7)

La etapa de agregación **\$unwind** genera un documento distinto por cada elemento que haya en el array indicado.

EXAM(2025J2.14)

La etapa de agregación **\$group** agrupa documentos en función de la condición indicada.

Es obligatorio establecer un valor **_id** para indicar bajo qué concepto se agrupa.

Equivale a **GROUP BY** de SQL.

=EXAM(2022J2.10)

La etapa de agregación **\$sort** ordena documentos en función del campo indicado.

Cuando el valor es 1 (positivo), el orden es ascendente mientras que cuando es -1 (negativo) el orden es descendente.

Equivale a **ORDER BY** de SQL.

La etapa de agregación **\$geoNear** ordena los documentos por proximidad, por lo que solo tiene sentido con datos geoespaciales.

La etapa de agregación **\$lookup** procesa una unión realizando un *left outer join* a otra colección en la misma base de datos.

Operadores de expresión:

- \$addFields
- \$avg
- \$gte
- \$filter
- \$size
- \$sum

=EXAM(2025J2.15)

El operador de expresión **\$addFields** añade uno o varios campos nuevos.

=EXAM(2025J2.15)

El operador de expresión **\$avg** aplica la media.

=EXAM(2025J2.15)

El operador de expresión **\$gte** selecciona resultados en base al criterio mayor o igual.

El operador de expresión **\$filter** filtra un array interno dentro de un documento. Cuando se usa **\$filter**, se mantiene el número de documentos original. No se explica en las diapositivas principales, pero ha aparecido en algún examen como opción errónea.

El operador de expresión **\$sum** suma el contenido de una variable, por ejemplo dentro de un **\$group**. **\$sum:"\$campo"** sería el equivalente a **SUM(campo)** en SQL.

\$sum se usa también para hacer conteos. La expresión idiomática **\$sum:1** sería el equivalente al **COUNT(*)** de SQL.

MapReduce

Índices en MongoDB

Los índices se actualizan cada vez que se realiza una operación de creación, actualización o borrado (CUD), por lo que añaden sobrecarga a la escritura.

Los índices aceleran las lecturas a cambio de ralentizar las escrituras, por lo que se recomienda su uso solamente cuando la agilización de lecturas supere la ralentización en escrituras, o cuando existan muchas lecturas y pocas escrituras.

La función **createIndex()** crea un índice

```
db.usuarios.createIndex({ nombre: 1 })
```

La propiedad **sparse** (disperso), cuando está activada, hace que el índice contenga solamente a los documentos que realmente tienen el campo indexado. Suele estar desactivado por defecto.

Tipos de índices en MongoDB:

- Índice simple
- Índice compuesto
- Índice multiclave
- Índice sobre texto
- Índice hash
- Índice para geoposicionamiento

Un **índice simple**...

Un **índice compuesto** (*compound index*) en MongoDB es un índice que se aplica a varios campos al mismo tiempo.

=EXAM(2024SO.15)

Un **índice multiclave** (*multikey index*) en MongoDB es un índice aplicado sobre un campo que contiene un array. En tal caso, se creará automáticamente un índice por cada uno de los elementos contenidos en el array.

Un **índice hash**...

Un **índice para geoposicionamiento**...

Arquitectura de clustering

mongos)

EXAM=(2023J2.13)

mongos (de *MongoDB sharding router*) es un proceso enrutador que se encarga de hacer *sharding* entre las máquinas MongoDB cuando se da la orden desde la aplicación.

Recibe órdenes de la aplicación y se encarga de decidir a qué *shard* debe ir cada parte de la consulta.

mongod)

EXAM=(2023J2.16)

mongod (de *MongoDB daemon*) es un proceso que gestiona individualmente cada máquina de bases de datos o *shard* de MongoDB, almacenando las consultas y gestionando datos.

Replica Set)

EXAM=(2024S0.14)

Un **replica set** es un grupo de instancias de **mongod** que proporciona redundancia y alta disponibilidad al alojar todos exactamente el mismo conjunto de datos.

Sigue una arquitectura primario-secundario, donde existe un único nodo primario, que realiza por defecto las lecturas y en todos los casos las escrituras, y varios secundarios, que solo realizan lecturas.

Un **árbitro** (*arbiter*) es un proceso **mongod** especial y ligero que no almacena una copia de datos, y que mantiene una única función en participar en elecciones para evitar empate. Se aplica cuando el número de instancias **mongod** es impar.

Cuando el nodo primario cae, los nodos secundarios votan para escoger un nuevo nodo primario. El límite de nodos con derecho a voto en un *replica set* es 7.

El máximo de miembros de un *replica set* es de 50.

Chunk)

Un **chunk** es un fragmento o bloque lógico de datos que MongoDB utiliza internamente cuando hace sharding para saber cómo repartir la colección entre los diferentes shards.

Sharding en MongoDB

Shard keys y chunks

Servidores de configuración

Requisitos mínimos de sharding en MongoDB EXAM=(2022J2.9)

Requisitos de un *sharded cluster* en MongoDB:

- Shards (Fragmentos): Cada uno se despliega idealmente como un *replica set* de 3 miembros para garantizar alta disponibilidad.
- *Config Servers* (Servidores de configuración): Un *replica set* de 3 miembros dedicado exclusivamente a almacenar los metadatos del clúster.
- Enrutadores (**mongos**): Uno o más servicios que actúan como interfaz para la aplicación. Su función es:
 - Enrutar operaciones de lectura y escritura al *shard* correcto.
 - Consolidar resultados mediante el uso de cursores distribuidos.
 - Cachear los metadatos de los *chunks* para acelerar el acceso a los datos.

Réplicas

Elección de primario

Lectura de datos

Estrategia de almacenamiento

Escritura de datos

Otras características

Licencia y atribución

License CC BY 4.0

Este trabajo está licenciado bajo la licencia **Creative Commons Atribución 4.0 Internacional**.

© 2026, Colaboradores de apuntes-muicd-uned.